

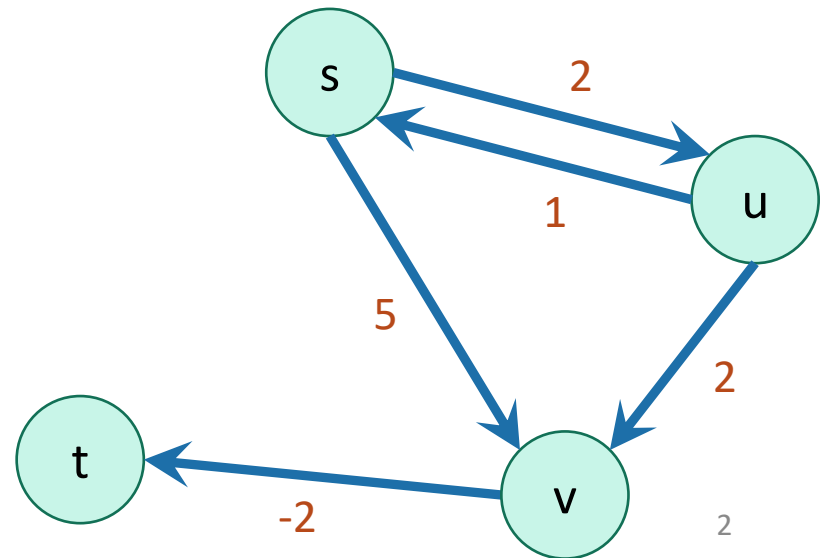
Floyd-Warshall

Floyd-Warshall Algorithm

Another example of DP

- This is an algorithm for **All-Pairs Shortest Paths (APSP)**
 - That is, I want to know the shortest path from u to v for **ALL** pairs u, v of vertices in the graph.
 - Not just from a special single source s .

Source	Destination				
	s	u	v	t	
	s	0	2	4	2
	u	1	0	2	0
	v	∞	∞	0	-2
	t	∞	∞	∞	0



Floyd-Warshall Algorithm

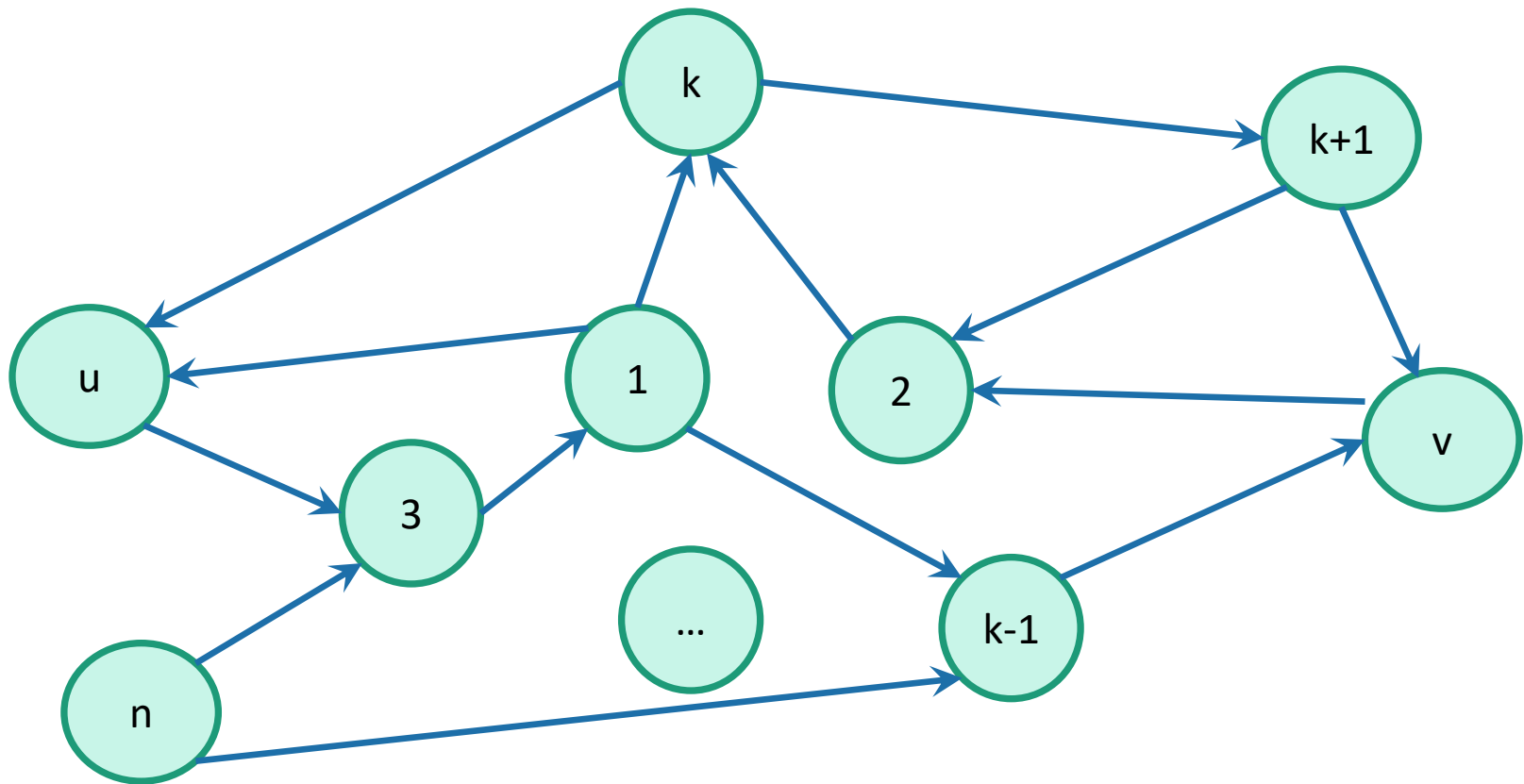
Another example of DP

- This is an algorithm for **All-Pairs Shortest Paths (APSP)**
 - That is, I want to know the shortest path from u to v for **ALL pairs** u, v of vertices in the graph.
 - Not just from a special single source s .
- Naïve solution (if we want to handle negative edge weights):
 - For all s in G :
 - Run Bellman-Ford on G starting at s .
 - Time $O(n \cdot nm) = O(n^2m)$,
 - may be as bad as n^4 if $m=n^2$

Can we do better?

Label the vertices $1, 2, \dots, n$

Optimal substructure



Optimal substructure

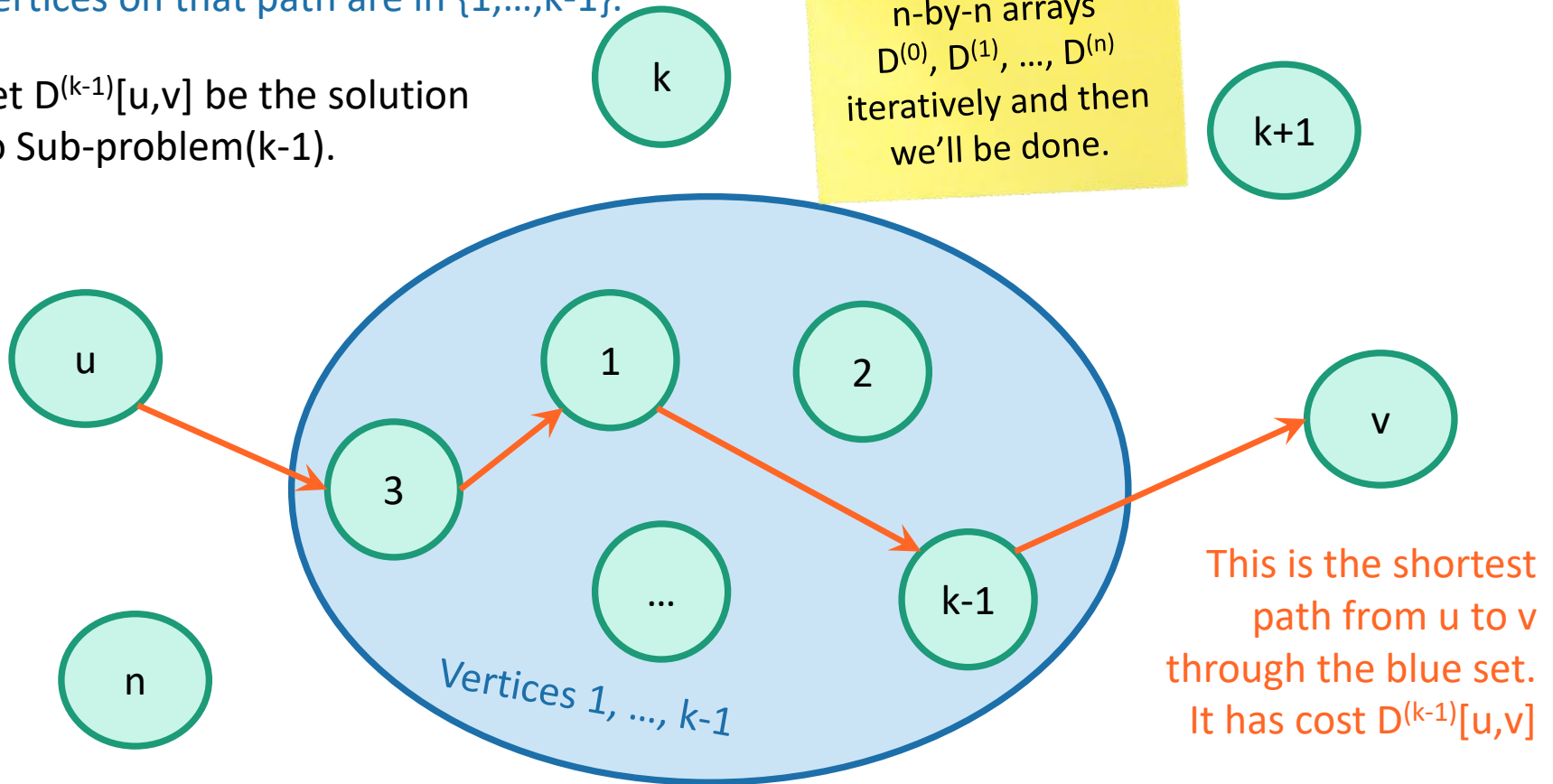
Label the vertices $1, 2, \dots, n$
(We omit some edges in the picture below – meant to be a cartoon, not an example).

Sub-problem($k-1$):

For all pairs, u, v , find the cost of the shortest path from u to v , so that all the internal vertices on that path are in $\{1, \dots, k-1\}$.

Let $D^{(k-1)}[u, v]$ be the solution to Sub-problem($k-1$).

Our DP algorithm will fill in the n -by- n arrays $D^{(0)}, D^{(1)}, \dots, D^{(n)}$ iteratively and then we'll be done.



Optimal substructure

Label the vertices $1, 2, \dots, n$
(We omit some edges in the picture below – meant to be a cartoon, not an example).

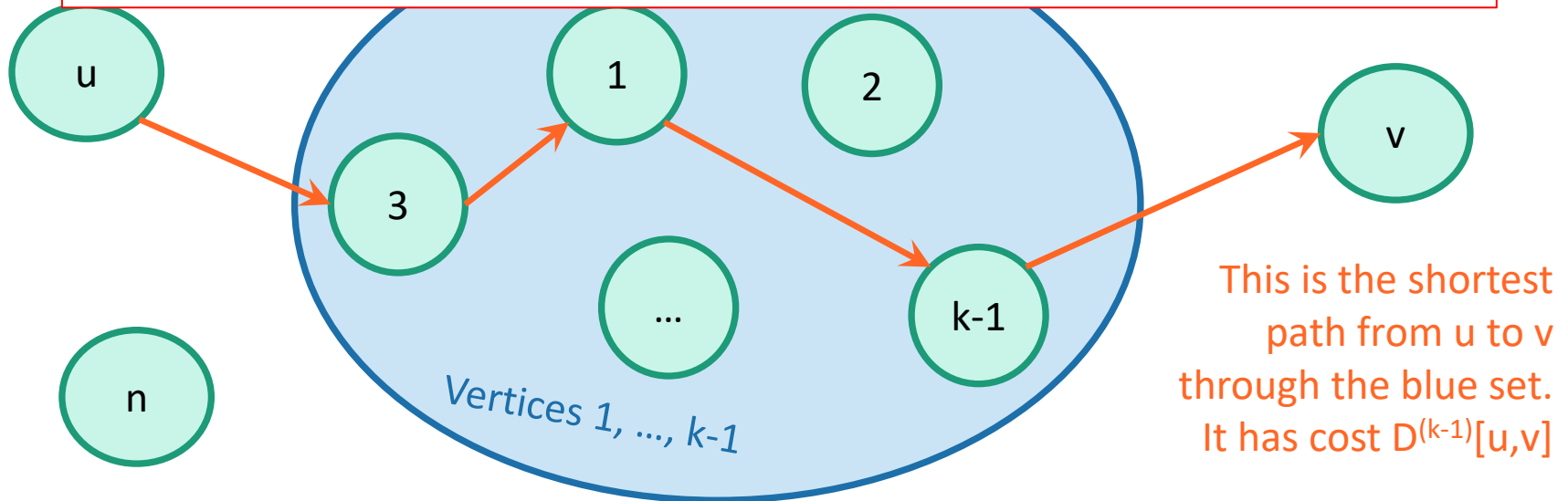
Sub-problem($k-1$):

For all pairs, u, v , find the cost of the shortest path from u to v , so that all the internal vertices on that path are in $\{1, \dots, k-1\}$.

Let $D^{(k-1)}[u, v]$ be the solution to Sub-problem($k-1$).

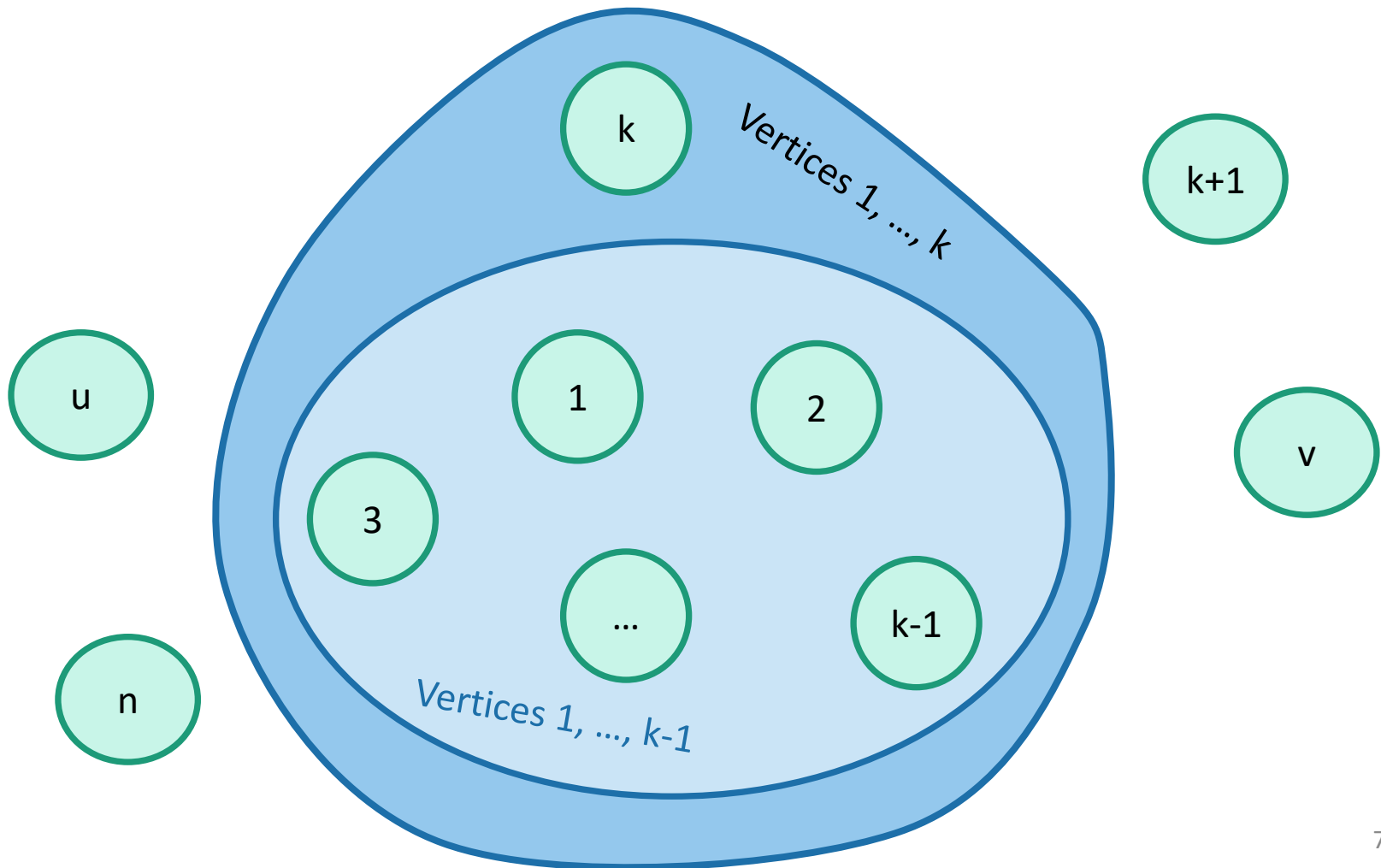
Our DP algorithm will fill in the n -by- n arrays $D^{(0)}, D^{(1)}, \dots, D^{(n)}$ iteratively and then we'll be done.

Question: How can we find $D^{(k)}[u, v]$ using $D^{(k-1)}$?



How can we find $D^{(k)}[u,v]$ using $D^{(k-1)}$?

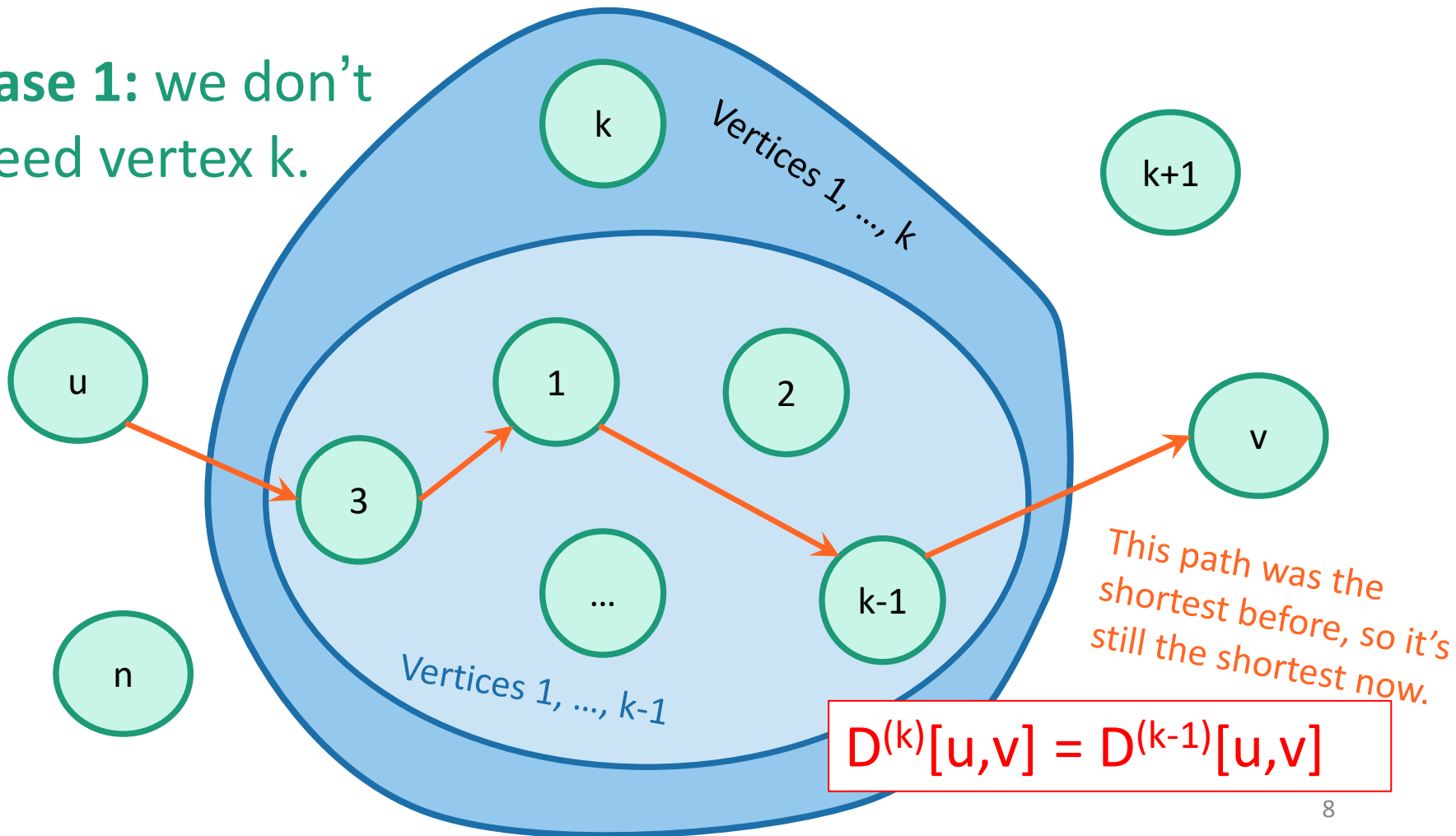
$D^{(k)}[u,v]$ is the cost of the shortest path from u to v so that all internal vertices on that path are in $\{1, \dots, k\}$.



How can we find $D^{(k)}[u,v]$ using $D^{(k-1)}$?

$D^{(k)}[u,v]$ is the cost of the shortest path from u to v so that all internal vertices on that path are in $\{1, \dots, k\}$.

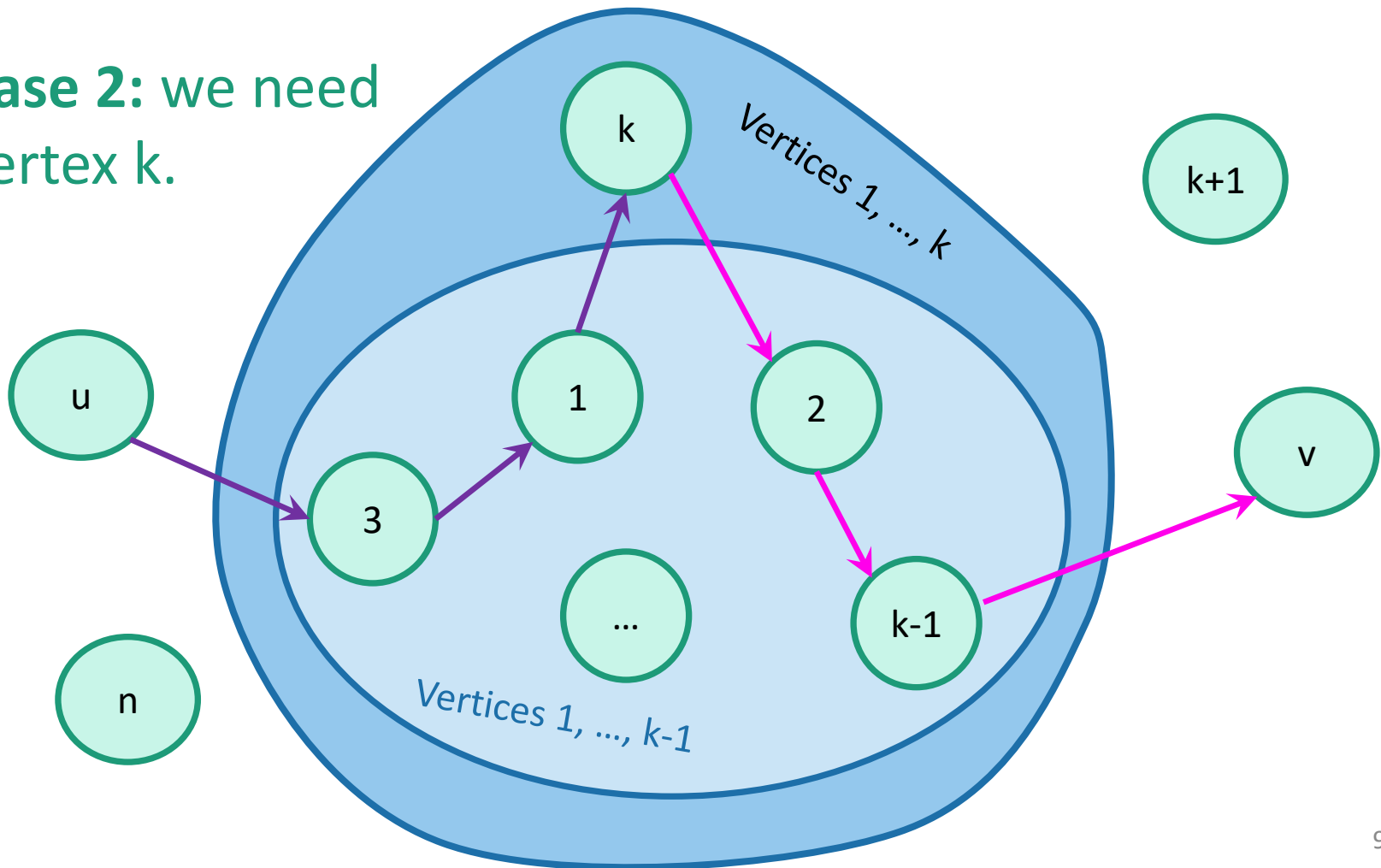
Case 1: we don't need vertex k .



How can we find $D^{(k)}[u,v]$ using $D^{(k-1)}$?

$D^{(k)}[u,v]$ is the cost of the shortest path from u to v so that all internal vertices on that path are in $\{1, \dots, k\}$.

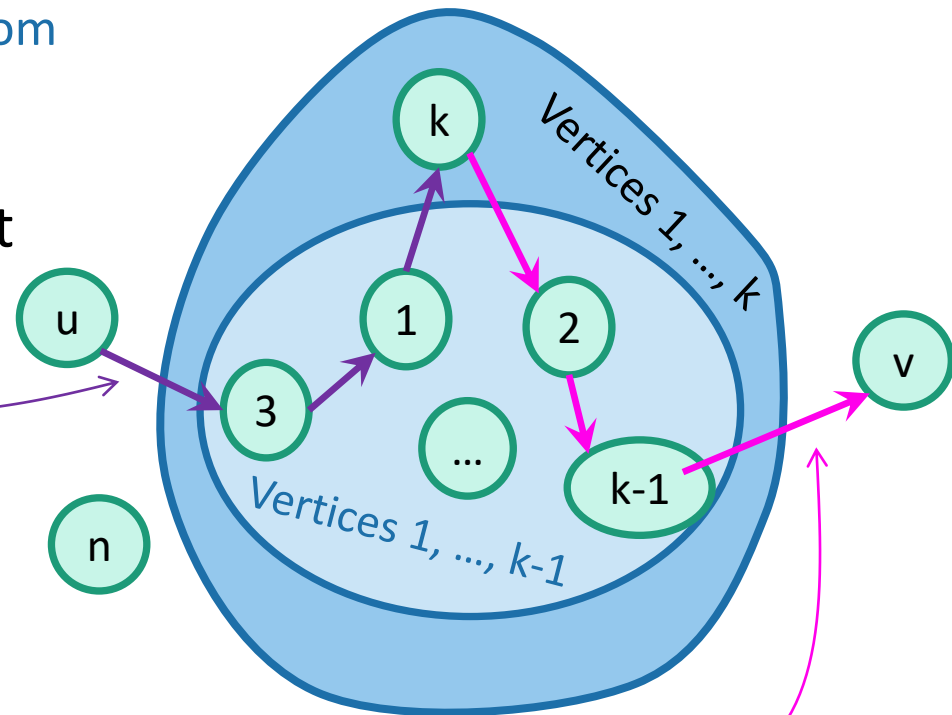
Case 2: we need vertex k .



Case 2 continued

- Suppose there are **no negative cycles**.
 - Then WLOG the shortest path from u to v through $\{1, \dots, k\}$ is **simple**.
- If that path passes through k , it must look like this: 
- This path is the shortest path from u to k through $\{1, \dots, k-1\}$.
 - sub-paths of shortest paths are shortest paths
- Similarly for this path.

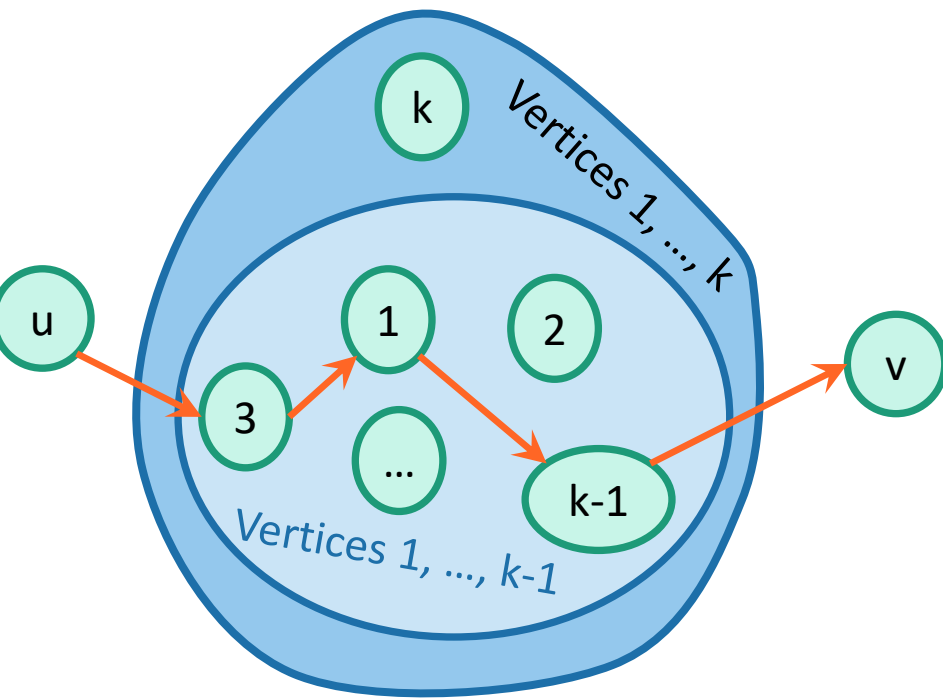
Case 2: we need vertex k .



$$D^{(k)}[u, v] = D^{(k-1)}[u, k] + D^{(k-1)}[k, v]_{10}$$

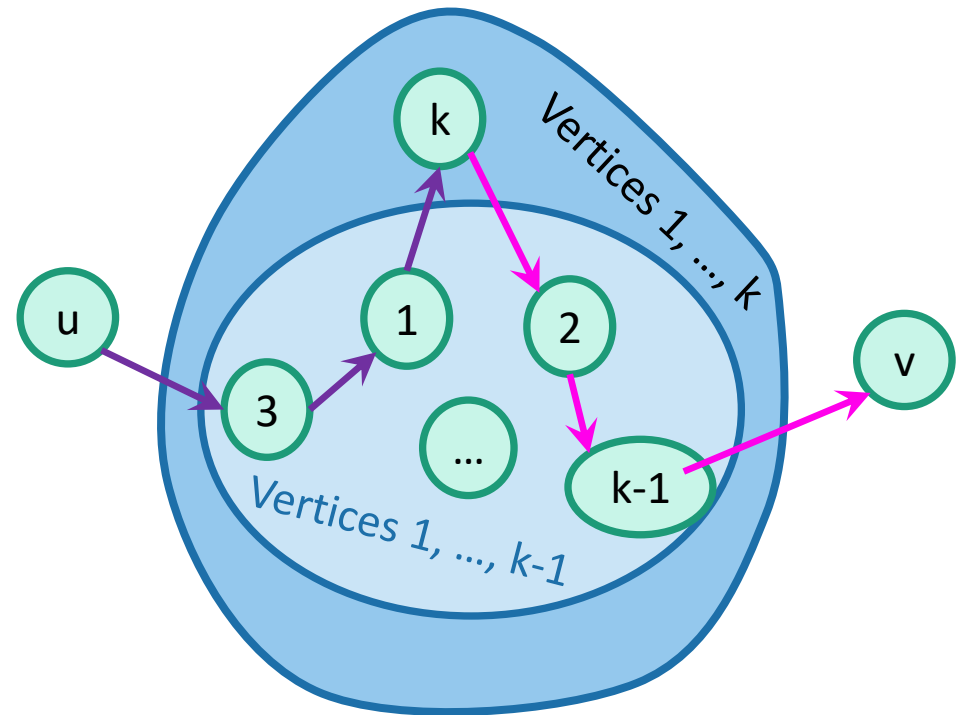
How can we find $D^{(k)}[u,v]$ using $D^{(k-1)}$?

Case 1: we don't need vertex k .



$$D^{(k)}[u,v] = D^{(k-1)}[u,v]$$

Case 2: we need vertex k .



$$D^{(k)}[u,v] = D^{(k-1)}[u,k] + D^{(k-1)}[k,v]$$

How can we find $D^{(k)}[u,v]$ using $D^{(k-1)}$?

- $D^{(k)}[u,v] = \min\{ D^{(k-1)}[u,v], D^{(k-1)}[u,k] + D^{(k-1)}[k,v] \}$

Case 1: Cost of
shortest path
through $\{1, \dots, k-1\}$

Case 2: Cost of shortest path
from **u** to **k** and then from **k** to **v**
through $\{1, \dots, k-1\}$

- Optimal substructure:
 - We can solve the big problem using solutions to smaller problems.
- Overlapping sub-problems:
 - $D^{(k-1)}[k,v]$ can be used to help compute $D^{(k)}[u,v]$ for lots of different u 's.

Floyd-Warshall algorithm

- Initialize n-by-n arrays $D^{(k)}$ for $k = 0, \dots, n$
 - $D^{(k)}[u,u] = 0$ for all u , for all k
 - $D^{(k)}[u,v] = \infty$ for all $u \neq v$, for all k
 - $D^{(0)}[u,v] = \text{weight}(u,v)$ for all (u,v) in E .
- **For** $k = 1, \dots, n$:
 - **For** pairs u,v in V^2 :
 - $D^{(k)}[u,v] = \min\{ D^{(k-1)}[u,v], D^{(k-1)}[u,k] + D^{(k-1)}[k,v] \}$
- **Return** $D^{(n)}$

The base case checks out: the only path through zero other vertices are edges directly from u to v .

This is a bottom-up *Dynamic programming* algorithm.

We've basically just shown

- Theorem:

If there are **no negative cycles** in a weighted directed graph G , then the Floyd-Warshall algorithm, running on G , returns a matrix $D^{(n)}$ so that:

$$D^{(n)}[u,v] = \text{distance between } u \text{ and } v \text{ in } G.$$

Work out the
details of a proof!



- Running time: $O(n^3)$

- Better than running Bellman-Ford n times!

- Storage:

- Need to store **two** n -by- n arrays, and the original graph.

As with Bellman-Ford, we don't really need to store all n of the $D^{(k)}$.

What if there *are* negative cycles?

- Just like Bellman-Ford, Floyd-Warshall can detect negative cycles:
 - “Negative cycle” means that there’s some v so that there is a path from v to v that has cost < 0 .
 - Aka, $D^{(n)}[v,v] < 0$.
- Algorithm:
 - Run Floyd-Warshall as before.
 - If there is some v so that $D^{(n)}[v,v] < 0$:
 - **return** negative cycle.

What have we learned?

- The Floyd-Warshall algorithm is another example of *dynamic programming*.
- It computes All Pairs Shortest Paths in a directed weighted graph in time $O(n^3)$.

Can we do better than $O(n^3)$?

Nothing on this slide is required knowledge for this class

- There is an algorithm that runs in time $O(n^3/\log^{100}(n))$.
 - *[Williams, “Faster APSP via Circuit Complexity”, STOC 2014]*
- If you can come up with an algorithm for All-Pairs-Shortest-Path that runs in time $O(n^{2.99})$, that would be a really big deal.
 - Let me know if you can!
 - See *[Abboud, Vassilevska-Williams, “Popular conjectures imply strong lower bounds for dynamic problems”, FOCS 2014]* for some evidence that this is a very difficult problem!